



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:

19/02/2026

Hora de inicio:

17:20

Software Avanzado Seccion B

Kevin Raúl Pozuelos Estrada

Arquitectura de Software

Software Avanzado

Escuela de Ingeniería de Ciencias Y Sistemas

Agenda



RECORDATORIOS



Dudas Iniciales



Anuncios Generales



Docker



Docker Compose



Vibe Coding



Dudas Finales

Docker: La magia de los
contenedores

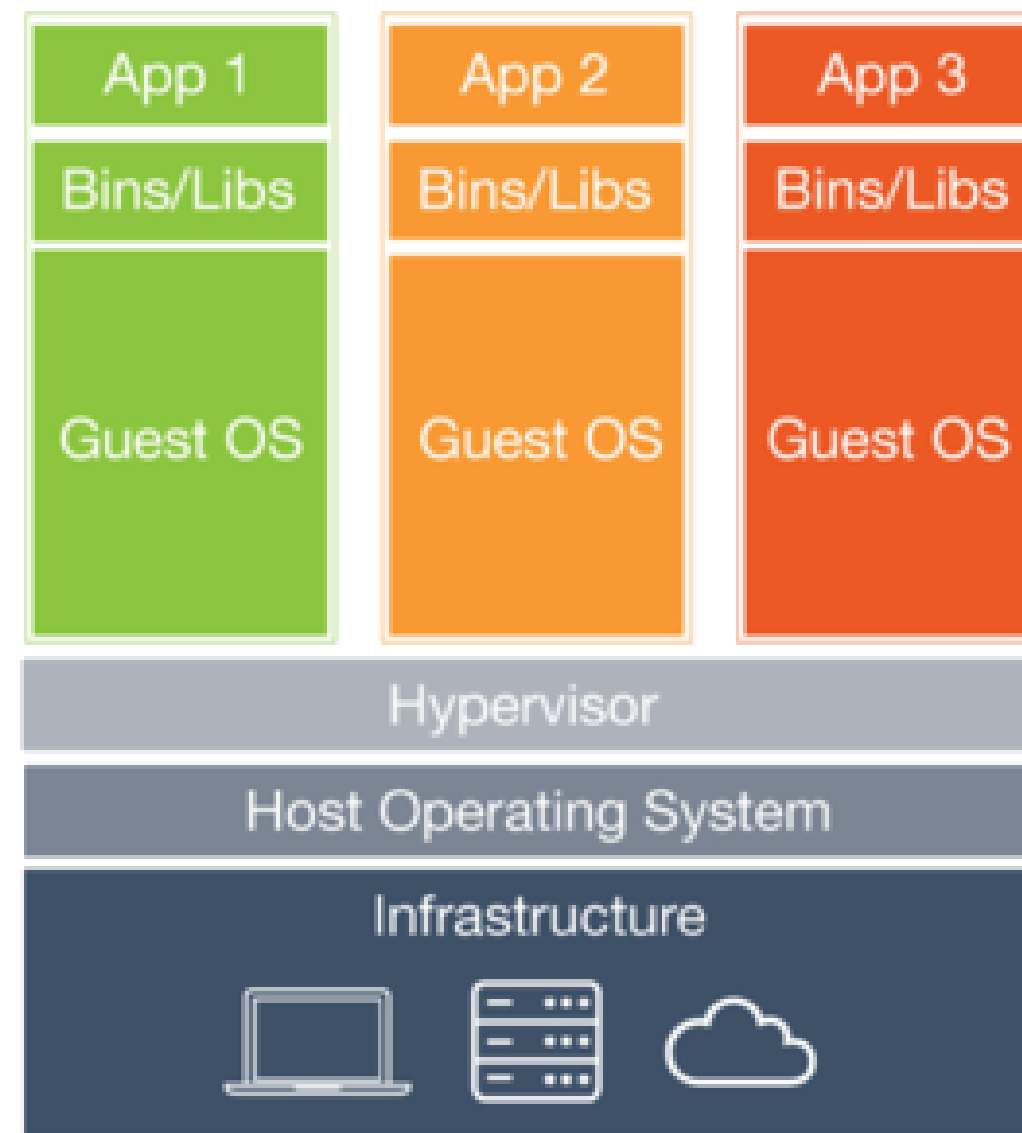


Plataforma de código abierto que
permite a los desarrolladores crear,
implementar, y gestionar
contenedores.

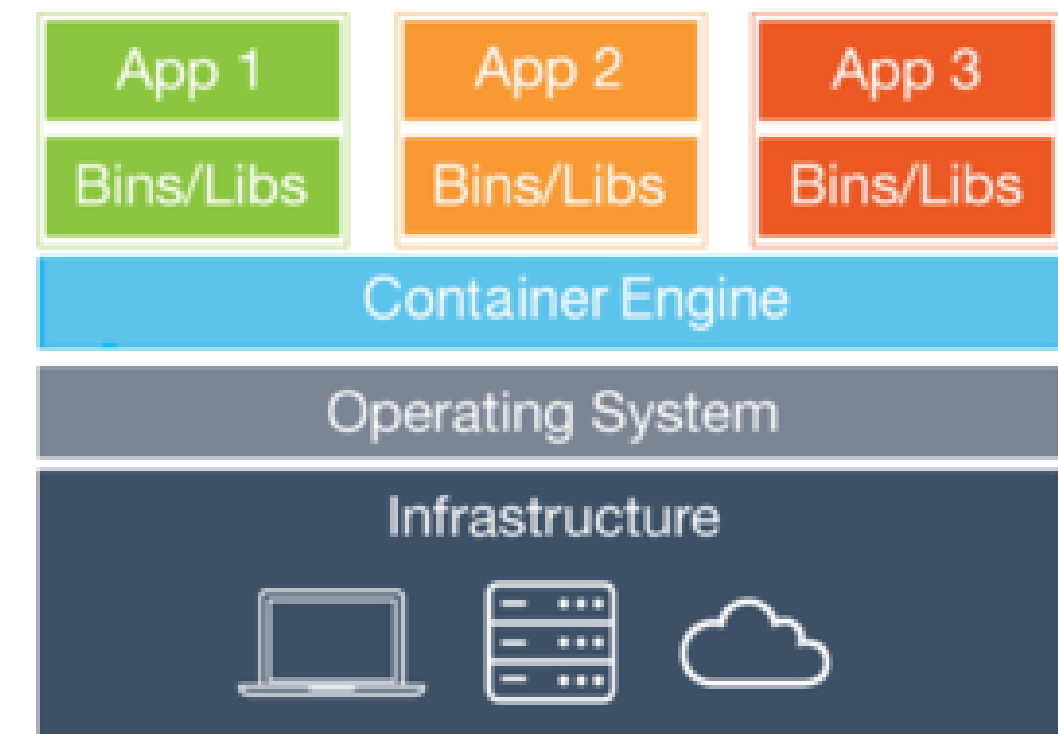
Unidades ejecutables de software donde se empaqueta el código de aplicación, junto con sus bibliotecas y dependencias, de forma común para que se pueda ejecutar en cualquier lugar

¿Qué son los contenedores?

Arquitectura de Docker



Hypervisor-based Virtualization

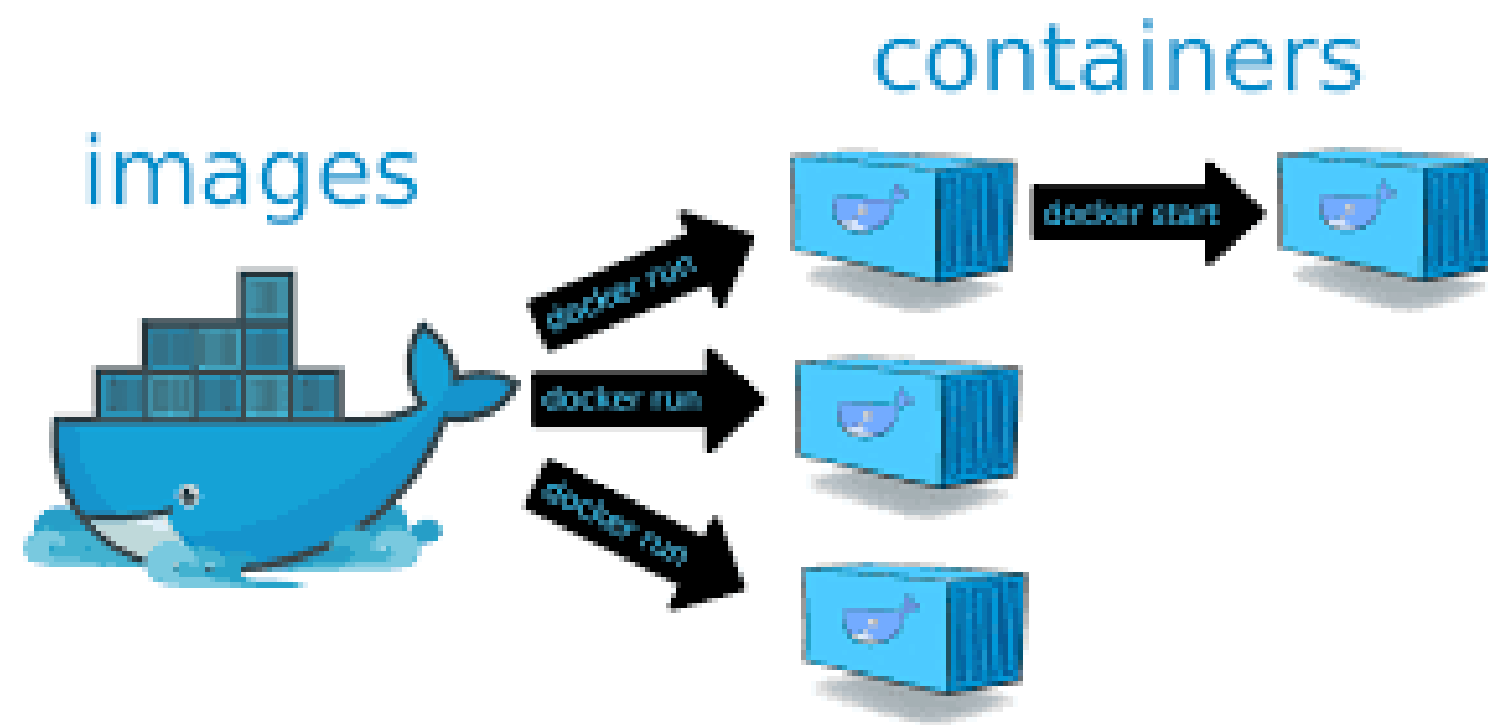


Container virtualization

Docker Image

Es una plantilla de solo lectura que define su contenedor. La imagen contiene el código que se ejecutará, incluida cualquier definición para cualquier biblioteca o dependencia que el código necesite.

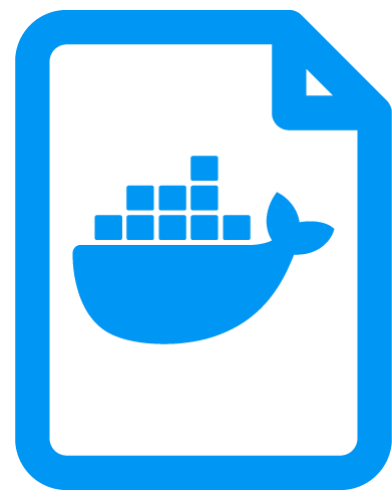




¿Es un contenedor lo mismo que una imagen?

Respuesta: No

Dockerfile: la forma de definir un contenedor



Archivo de texto que contiene las instrucciones necesarias para crear una nueva imagen del contenedor.

Estas instrucciones incluyen la identificación de una imagen existente que se usará como base, los comandos que se ejecutarán durante el proceso de creación de la imagen y un comando que se ejecutará cuando se implementen instancias nuevas de la imagen del contenedor.

```
# syntax=docker/dockerfile:1
```

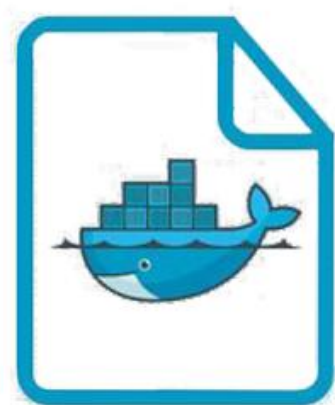
```
FROM ubuntu:22.04
```

```
COPY . /app
```

```
RUN make /app
```

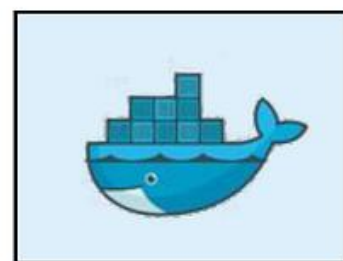
```
CMD python /app/app.py
```

Ejemplo de Dockerfile



Dockerfile

Build →



Docker
Image

Run →



Docker
Container

Flujo de un contenedor
de docker

Como desplegar un contenedor

Comando	Explicación
docker build	Construye una imagen a partir de un Dockerfile en el directorio actual
docker build https://github.com/docker/rootfs.git#contenedor:docker	Construye una imagen desde un repositorio GIT remoto
docker build -t imagename/tag	Construye y etiqueta una imagen para facilitar su seguimiento
docker build https://yourserver/file.tar.gz	Crea una imagen a partir de un archivo tar remoto
docker build -t imagen:1.0-<<EOFFFROM busyboxRUN echo «hola mundo «EOF	Construye una imagen mediante un archivo Dockerfile que se pasa a través de STDIN



Cheatsheet for Docker CLI

Run a new Container

Start a new Container from an Image

```
docker run IMAGE
```

```
docker run nginx
```

...and assign it a name

```
docker run --name CONTAINER IMAGE
```

```
docker run --name web nginx
```

...and map a port

```
docker run -p HOSTPORT:CONTAINERPORT IMAGE
```

```
docker run -p 8080:80 nginx
```

...and map all ports

```
docker run -P IMAGE
```

```
docker run -P nginx
```

...and start container in background

```
docker run -d IMAGE
```

```
docker run -d nginx
```

...and assign it a hostname

```
docker run --hostname HOSTNAME IMAGE
```

```
docker run --hostname srv nginx
```

...and add a dns entry

```
docker run --add-host HOSTNAME:IP IMAGE
```

...and map a local directory into the container

```
docker run -v HOSTDIR:TARGETDIR IMAGE
```

```
docker run -v ~/.:/usr/share/nginx/html nginx
```

...but change the entrypoint

```
docker run -it --entrypoint EXECUTABLE IMAGE
```

```
docker run -it --entrypoint bash nginx
```

Manage Containers

Show a list of running containers

```
docker ps
```

Show a list of all containers

```
docker ps -a
```

Delete a container

```
docker rm CONTAINER
```

```
docker rm web
```

Delete a running container

```
docker rm -f CONTAINER
```

```
docker rm -f web
```

Delete stopped containers

```
docker container prune
```

Stop a running container

```
docker stop CONTAINER
```

```
docker stop web
```

Start a stopped container

```
docker start CONTAINER
```

```
docker start web
```

Copy a file from a container to the host

```
docker cp CONTAINER:SOURCE TARGET
```

```
docker cp web:/index.html index.html
```

Copy a file from the host to a container

```
docker cp TARGET CONTAINER:SOURCE
```

```
docker cp index.html web:/index.html
```

Start a shell inside a running container

```
docker exec -it CONTAINER EXECUTABLE
```

```
docker exec -it web bash
```

Rename a container

```
docker rename OLD_NAME NEW_NAME
```

```
docker rename 096 web
```

Create an image out of container

```
docker commit CONTAINER
```

```
docker commit web
```

Manage Images

Download an image

```
docker pull IMAGE[:TAG]
```

```
docker pull nginx
```

Upload an image to a repository

```
docker push IMAGE
```

```
docker push myimage:1.0
```

Delete an image

```
docker rmi IMAGE
```

Show a list of all Images

```
docker images
```

Delete dangling images

```
docker image prune
```

Delete all unused images

```
docker image prune -a
```

Build an image from a Dockerfile

```
docker build DIRECTORY
```

```
docker build .
```

Tag an image

```
docker tag IMAGE NEWIMAGE
```

```
docker tag ubuntu ubuntu:18.04
```

Build and tag an image from a Dockerfile

```
docker build -t IMAGE DIRECTORY
```

```
docker build -t myimage .
```

Save an image to .tar file

```
docker save IMAGE > FILE
```

```
docker save nginx > nginx.tar
```

Load an image from a .tar file

```
docker load -i TARFILE
```

```
docker load -i nginx.tar
```

Info & Stats

Show the logs of a container

```
docker logs CONTAINER
```

```
docker logs web
```

Show stats of running containers

```
docker stats
```

Show processes of container

```
docker top CONTAINER
```

```
docker top web
```

Show installed docker version

```
docker version
```

Get detailed info about an object

```
docker inspect NAME
```

```
docker inspect nginx
```

Show all modified files in container

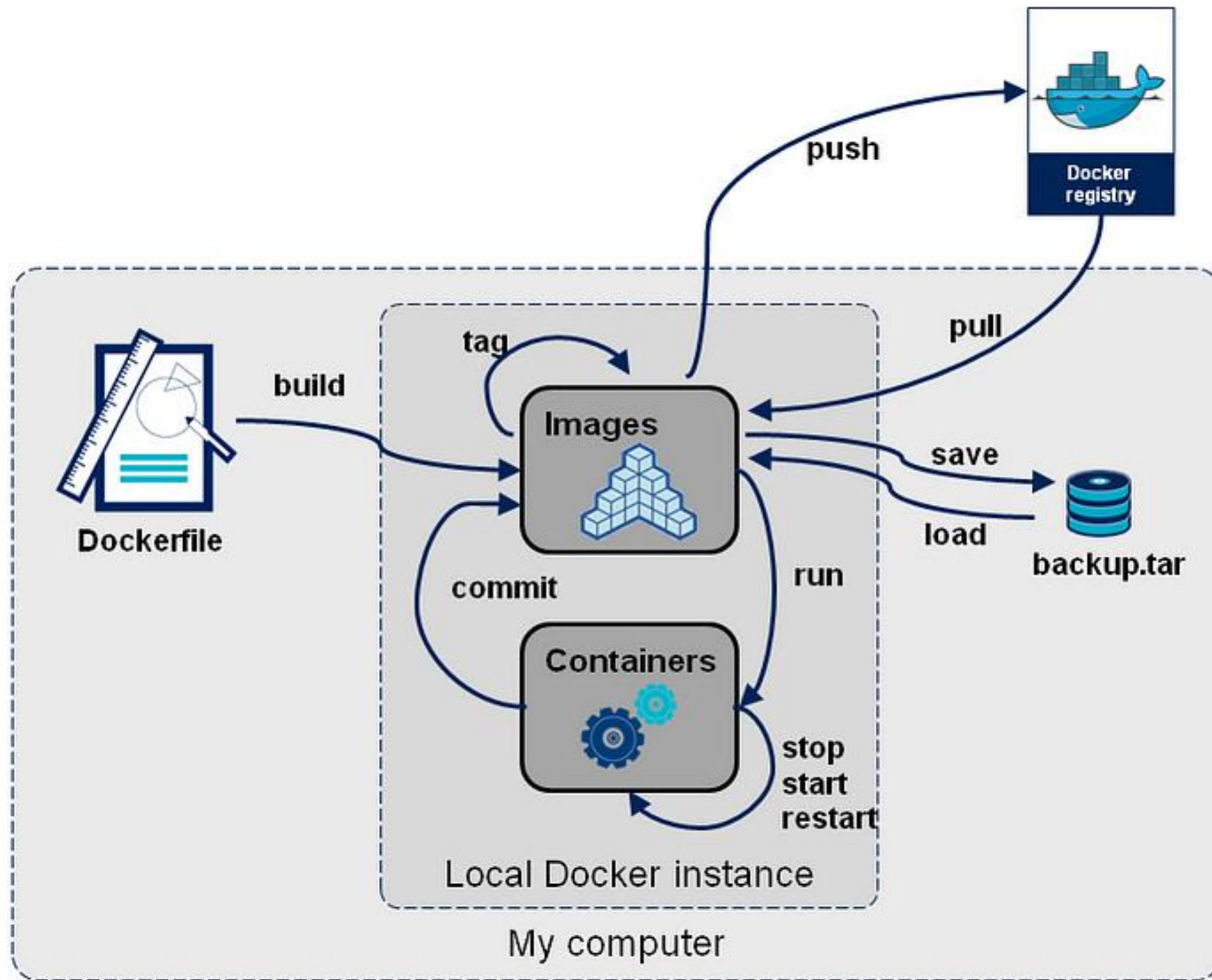
```
docker diff CONTAINER
```

```
docker diff web
```

Show mapped ports of a container

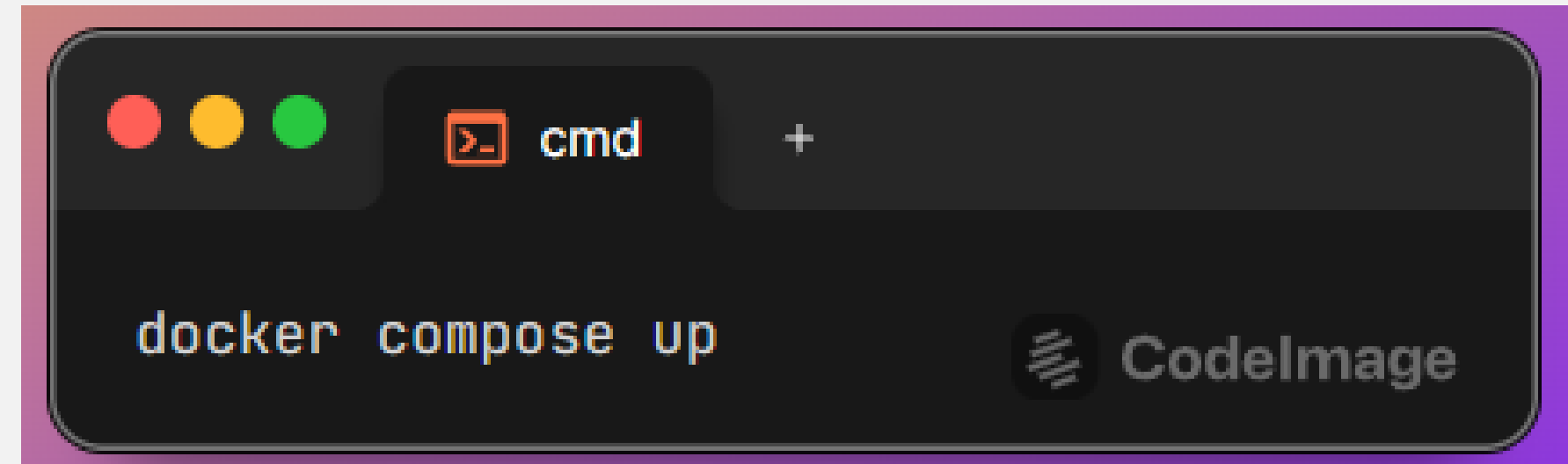
```
docker port CONTAINER
```

```
docker port web
```



Ciclo de vida de un contenedor

¿Qué es Docker Compose?



-
- **Docker Compose** es una herramienta que permite **definir y ejecutar aplicaciones multicontenedor** usando un solo archivo YAML (docker-compose.yml).

Es ideal
para



¿Por qué usar Docker Compose?

Problema sin Compose:

Si tienes:

API en Node.js

Base de datos PostgreSQL

Redis para caché

Tendrías que ejecutar 3
comandos largos y
configurarlos manualmente.

Solución con Compose:

Defines todo en un archivo y
levantas todo con un solo
comando.

- ✓ Automatiza configuración
- ✓ Evita errores manuales
- ✓ Facilita trabajo en equipo
- ✓ Permite entornos reproducibles

Conceptos clave

Elemento	Significado
<code>services</code>	Contenedores que forman la aplicación
<code>image</code>	Imagen de Docker a usar
<code>ports</code>	Puertos expuestos
<code>volumes</code>	Persistencia de datos
<code>depends_on</code>	Orden de inicio
<code>environment</code>	Variables de entorno

Arquitectura típica con Compose

Docker Compose
automáticamente:

Crea una **red**
interna

Permite
comunicación por
nombre de
servicio

Maneja
dependencias

Gestiona
volúmenes

Comandos más importantes

Comando	Función
<code>docker compose up</code>	Levanta servicios
<code>docker compose up -d</code>	Levanta en segundo plano
<code>docker compose down</code>	Detiene y elimina contenedores
<code>docker compose build</code>	Construye imágenes
<code>docker compose ps</code>	Lista contenedores
<code>docker compose logs</code>	Ver logs

Ventajas

- ✓ Fácil de usar
- ✓ Ideal para desarrollo
- ✓ Configuración declarativa
- ✓ Portabilidad
- ✓ Control de versiones del entorno

Diferencia entre Docker y Docker Compose

Docker

Maneja un contenedor

`docker run`

Configuración manual

Docker Compose

Maneja múltiples contenedores

`docker compose up`

Configuración centralizada

El término "vibe coding" se ha popularizado para describir una nueva y emergente práctica de desarrollo de software que se apoya fundamentalmente en la inteligencia artificial (IA) para generar código funcional a partir de instrucciones en lenguaje natural. En esencia, es un enfoque en el que el desarrollador, más que escribir líneas de código de manera meticulosa, guía a un modelo de lenguaje grande (LLM) a través de una conversación para construir una aplicación o resolver un problema de programación.

¿Qué es el vibe coding?

Entendiendo la IA para darle un mejor uso

Para poder usar de mejor manera las herramientas, debemos de comprender como funcionan. Preguntas como: que es la IA, que es un LLM y como funcionan? nos llevaran a una mejor comprension de estas herramientas.



Espacio para dudas



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA